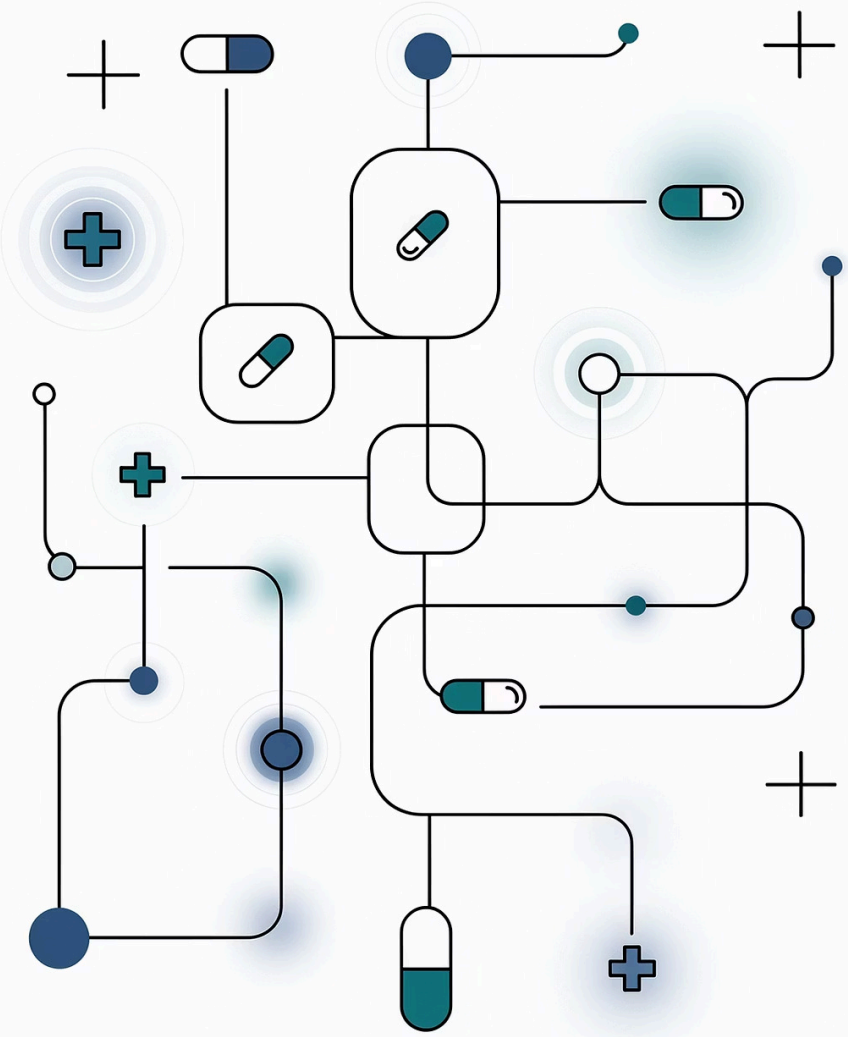
HL7 FHIR | IMPLEMENTATION GUIDES | INTEROPERABILITY

Getting Started with HL7 FHIR Implementation Guides

A practical primer on authoring FHIR Implementation Guides — from profiling resources to publishing conformance packages. Whether you're a standards architect, clinical informaticist, or integration engineer, this guide covers the core concepts, tooling, and step-by-step workflow you need to start building IGs with confidence.



Why Implementation Guides Matter

HL7 FHIR provides a flexible foundation for healthcare data exchange — but flexibility without governance leads to fragmentation. That's where **Implementation Guides (IGs)** come in. They are the critical layer that transforms FHIR's universal building blocks into **specific, actionable blueprints** for real-world interoperability.

The "House Rules" for Data Exchange

IGs define how FHIR resources are tailored for specific use cases — such as US Core for domestic EHR interoperability, or mCODE for oncology data. They provide the **"house rules"** that ensure every participant in a data exchange speaks the same dialect. Without these shared constraints, a Patient resource from System A may look wildly different from one in System B, even though both are technically "valid" FHIR.

Ensuring Interoperability at Scale

IGs guarantee that when a payer requests a clinical summary, or when a lab sends results to an EHR, every system agrees on **which fields are required, what terminologies to use, and how data is structured**. This is what transforms theoretical interoperability into practical, vendor-neutral data exchange that works across organizational and geographic boundaries.

The Cost of Going Without

Without Implementation Guides, healthcare IT implementations risk **fragmentation and incompatibility**. Each vendor invents its own conventions, leading to costly point-to-point integrations, mapping errors, and data loss. Regulatory programs like ONC's HTI-1 rule and CMS Interoperability mandates increasingly reference published IGs as the basis for certification and compliance — making IG literacy a professional necessity, not a luxury.

Key Takeaway: An Implementation Guide is the contract between systems — turning FHIR's open specification into a precise, testable agreement that vendors, regulators, and clinicians can rely on.

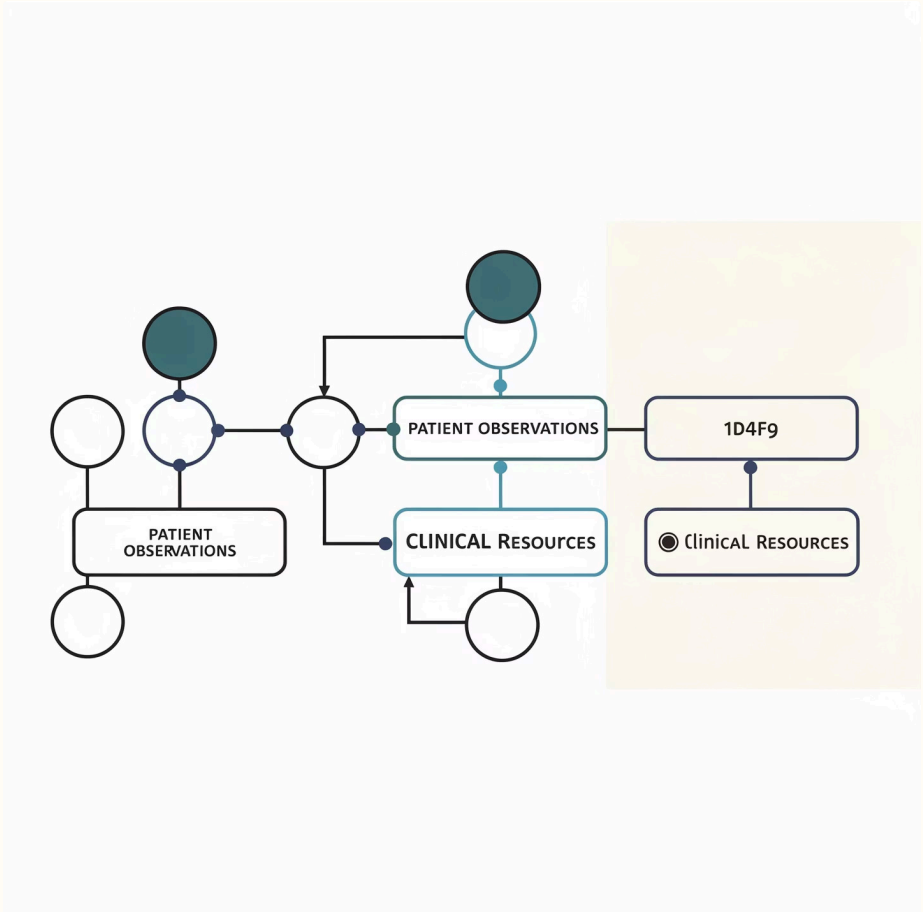
Core Concepts: Profiling FHIR Resources

At the heart of every Implementation Guide lies the concept of **profiling**. A FHIR profile is a set of constraints and extensions applied on top of a base FHIR resource — such as Patient, Observation, Condition, or MedicationRequest — to tailor it for a particular clinical or administrative domain. Profiling is how you narrow FHIR's intentionally broad specification into a precise, use-case-specific data model.

What Profiles Define

Profiles give IG authors fine-grained control over how resources are used. Key constraint types include:

- **Cardinality changes** — Make optional elements required (e.g., tighten `Patient.birthDate` from 0..1 to 1..1) or suppress elements by setting max to 0.
- **Fixed and pattern values** — Lock specific fields to predetermined values, such as requiring `Observation.code` to always carry a particular LOINC code for blood pressure readings.
- **Terminology bindings** — Bind coded elements to specific ValueSets with required, extensible, or preferred binding strength — ensuring consistent use of SNOMED CT, LOINC, RxNorm, or custom code systems.
- **Slicing** — Define specific patterns for repeating elements (e.g., requiring both a systolic and diastolic component inside a blood pressure Observation).
- **Extensions** — Add custom data elements not present in the base resource, such as race/ethnicity fields on Patient per US Core requirements.



Real-World Example: A cardiovascular Implementation Guide might profile the `Observation` resource to require LOINC code `85354-9` (blood pressure panel), mandate both systolic and diastolic components with units of `mm[Hg]`, and bind the interpretation field to a restricted ValueSet. This ensures every blood pressure reading across participating systems is structurally identical and semantically unambiguous.

The Profiling Hierarchy

Base Resource

The canonical FHIR resource definition (e.g., `Observation`) with all optional and required elements as defined in the specification.

Derived Profile

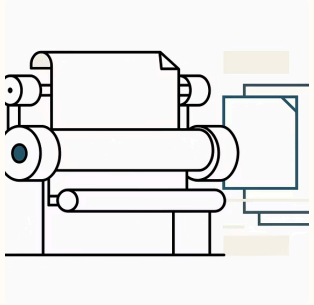
Your IG's constrained version — adding required bindings, cardinality changes, slicing rules, and extensions to fit your use case.

Conformant Instance

An actual resource instance that validates against your profile — ready for exchange between systems implementing your IG.

Tools of the Trade: FHIR IG Publisher & Sushi

Authoring an Implementation Guide requires more than clinical knowledge — you need a robust toolchain. The HL7 community has converged on two primary tools that form the backbone of modern IG development.



FHIR IG Publisher

The **IG Publisher** is the official HL7 tool that compiles your Implementation Guide source files into a **standardized, navigable website**. It processes StructureDefinitions, ValueSets, CodeSystems, examples, and narrative pages, rendering them into a browsable HTML publication that conforms to the HL7 publishing template. It also runs **FHIR validation** on all your resources and examples, catching errors before you publish. The IG Publisher is a Java application maintained by Grahame Grieve and the HL7 tooling team, and it's the same tool used to publish US Core, IPA, mCODE, and virtually every major FHIR IG in the ecosystem.

Sushi (FHIR Shorthand Compiler)

FHIR Shorthand (FSH) is a purpose-built domain-specific language that makes writing profiles, extensions, value sets, and examples far easier than hand-editing raw JSON or XML StructureDefinitions. **SUSHI ("SUSHI Unshortens Short Hand Inputs")** is the reference compiler that transforms `.fsh` files into the JSON artifacts the IG Publisher consumes. FSH reads like structured pseudocode — intuitive enough that clinical informaticists can author constraints directly, while being precise enough for machine compilation. It has become the de facto standard for IG authoring in the FHIR community, supported by comprehensive documentation at **fshschool.org** and a growing library of community examples.

Prerequisites & Dependencies

1

Java 11+ JDK

Required by the IG Publisher. Set your `JAVA_HOME` environment variable to the JDK installation path.

2

Node.js & npm

Sushi is distributed as an npm package. Install Node.js 18+ and then install Sushi globally via `npm install -g fsh-sushi`.

3

Ruby & Jekyll

The IG Publisher uses Jekyll for static site generation of the narrative pages and navigation templates.

4

Git

Version control is essential for collaborative IG development and for fetching IG Publisher updates and dependent IG packages.

Step 1: Set Up Your Development Environment

Before you author a single profile, you need a properly configured development environment. This section walks through the essential setup to get your workstation ready for IG authoring. While the toolchain has multiple moving parts, once configured it provides a smooth, repeatable workflow.

1 Install Java 11 JDK & Configure JAVA_HOME

Download and install the **Java 11+ JDK** (Eclipse Temurin or Oracle). After installation, set the `JAVA_HOME` environment variable to your JDK path. On macOS/Linux, add `export JAVA_HOME=$(/usr/libexec/java_home -v 11)` to your shell profile. On Windows, set it via System Properties → Environment Variables. Verify with `java -version` in your terminal — you should see version 11.x.x.

2 Clone the FHIR IG Publisher

Clone the IG Publisher from the official HL7 GitHub repository: `git clone https://github.com/HL7/fhir-ig-publisher.git`. Alternatively, download the latest `publisher.jar` directly from the HL7 Confluence page. Many IG projects include a `_updatePublisher.sh` (or `.bat`) script that automatically fetches the latest publisher JAR — run this regularly to stay current with bug fixes and features.

3 Install Ruby & Jekyll

The IG Publisher relies on **Jekyll** for static site generation, which requires Ruby 3.1+. On macOS, use `brew install ruby`; on Windows, use the RubyInstaller with DevKit. After Ruby is installed, run `gem install jekyll`. Confirm your installation with `jekyll -v`. Note: Some developers prefer using Docker-based IG Publisher images to bypass local Ruby/Jekyll configuration entirely.

4 Install Sushi (FHIR Shorthand Compiler)

Install Node.js (LTS version recommended), then run `npm install -g fsh-sushi`. Verify with `sushi -v`. Visit **fshschool.org** for comprehensive installation guidance, troubleshooting tips, and tutorials. Also consider installing the **FSH language extension** for VS Code, which provides syntax highlighting, auto-completion, and inline validation for `.fsh` files.

✅ Environment Verification Checklist

Run these commands to confirm everything is working:

- `java -version` → 11.x.x
- `ruby -v` → 3.1+
- `jekyll -v` → 4.x
- `sushi -v` → 3.x
- `git --version` → 2.x+

💡 Pro Tips

- Use **VS Code** as your IDE with the FSH and FHIR extensions for the best authoring experience.
- Consider **asdf** or **sdkman** for managing multiple Java versions across projects.
- Set up a `Makefile` or shell script to automate the Sushi → IG Publisher → Build pipeline into a single command.
- Join the **chat.fhir.org** Zulip community for real-time help from IG authors worldwide.

Step 2: Author Your Profiles and Extensions

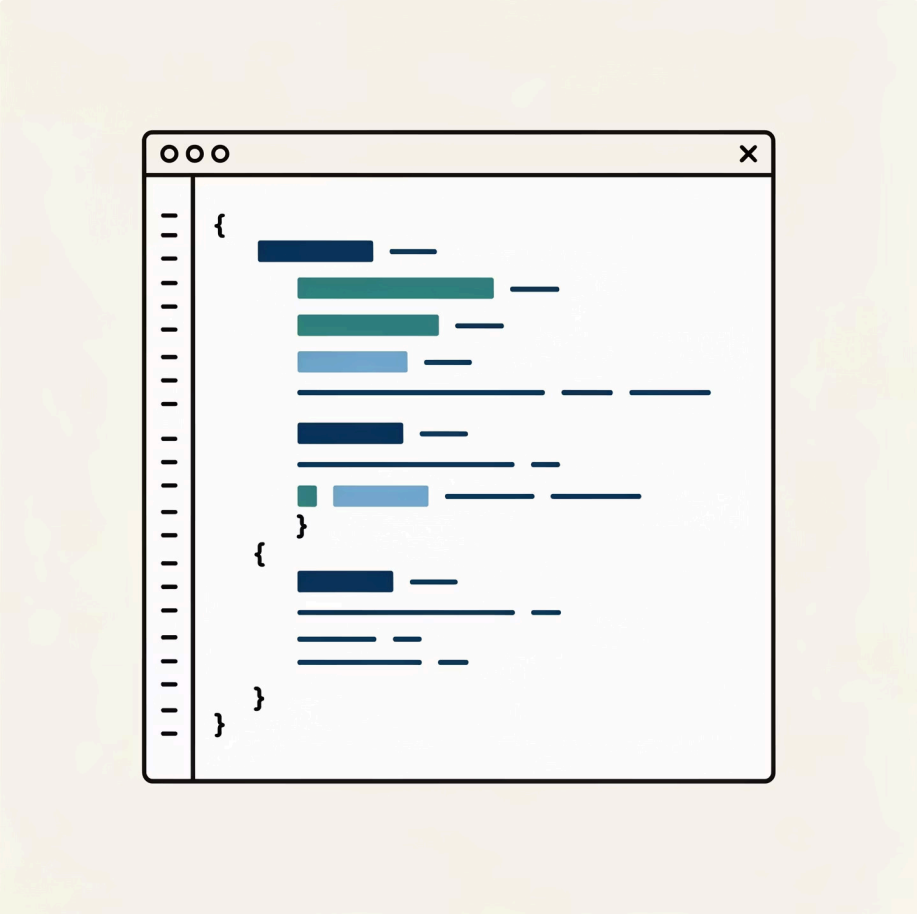
With your environment configured, it's time to write the FSH (FHIR Shorthand) files that define your IG's profiles, extensions, value sets, and examples. FSH provides an expressive, human-readable syntax that compiles into the JSON StructureDefinitions the IG Publisher consumes.

Writing Your First FSH Profile

Create a new `.fsh` file in your project's `input/fsh` directory. Each FSH file typically defines one or more profiles, extensions, or value sets. Here's an annotated example of a simple Patient profile:

```
Profile: MyPatientProfile
Parent: Patient
Id: my-patient-profile
Title: "My Patient Profile"
Description: "A constrained Patient for our IG."

// Make name required (1..*)
* name 1..* MS

// Require birthDate
* birthDate 1..1 MS

// Bind gender to required ValueSet
* gender 1..1 MS

// Add a custom extension
* extension contains
    PatientNationality named nationality 0..1

// Suppress unused elements
* photo 0..0
* contact 0..0
```

Each line maps to a constraint on the underlying StructureDefinition. The MS flag marks elements as Must Support — signaling that these fields must be populated when available and understood by receiving systems.

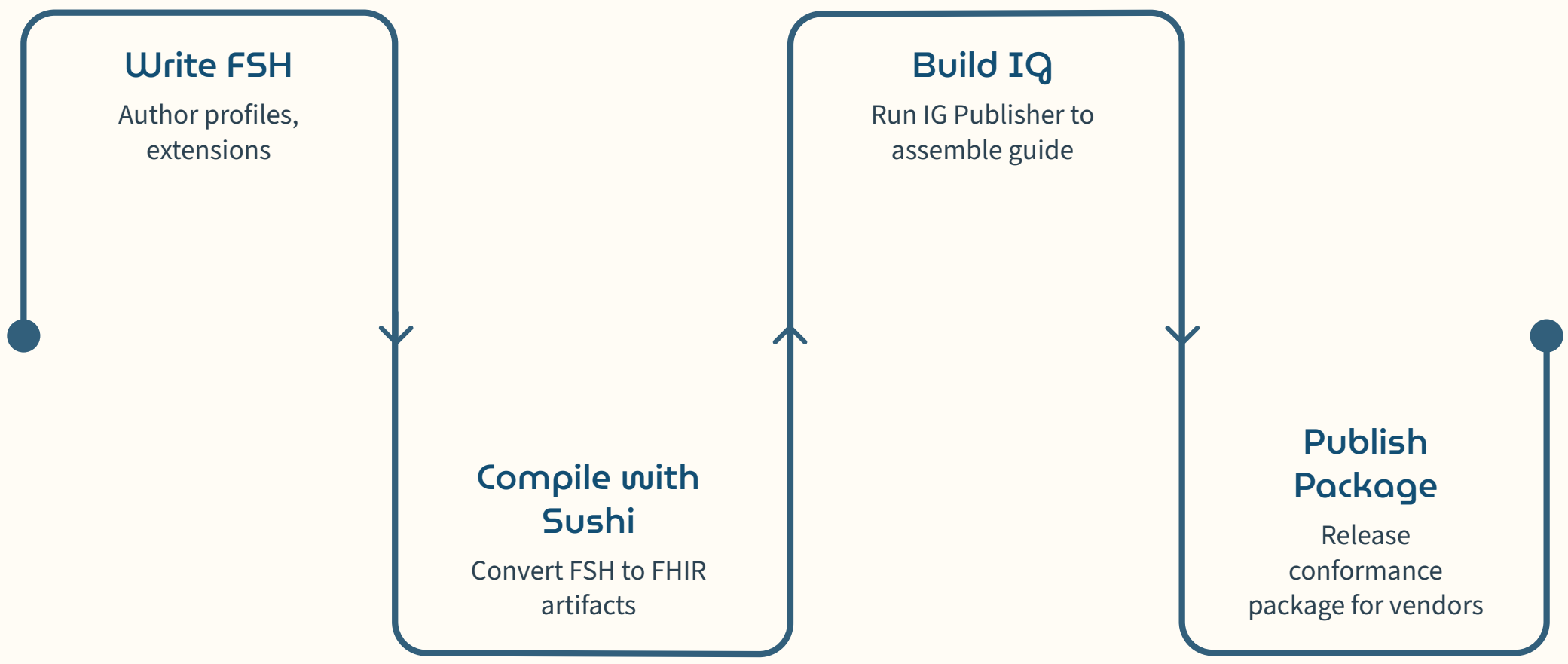


Compilation Workflow: Run `sushi .` from your project root to compile all `.fsh` files into JSON artifacts in the `fsh-generated` directory. Then run the IG Publisher with `java -jar publisher.jar -ig ig.ini` to build the full IG website — complete with validation results, rendered profiles, and downloadable conformance packages (NPM tarballs).

Key Authoring Best Practices

- **Start with use cases** — Define your clinical scenarios before writing constraints. Profiles should serve real data exchange needs.
- **Use invariants** — Add FHIRPath constraints for complex business rules (e.g., "if status is 'final', value must be present").
- **Provide examples** — Include valid and invalid example instances to help implementers understand expectations.
- **Version control** — Commit early and often. Use semantic versioning for your IG releases.
- **Test continuously** — Run the IG Publisher frequently during development; don't wait until the end to discover validation errors.

From Authoring to Adoption: The Publication Pipeline



Once compiled and validated, the IG Publisher generates an NPM conformance package for server-side validation, a browsable HTML site for human review, and downloadable bundles of StructureDefinitions, ValueSets, and CapabilityStatements. Submit your IG to the **FHIR IG Registry** and the **HL7 Package Registry** to make it discoverable and installable by the broader community.

Get in Touch

Email

info@aiternahealth.com

Phone

+1 (416) 262-3968

LinkedIn

aiterna-health-inc